API Apartamentos

Arquitectura orientada a servicios
Descripción del proyecto y evidencias de pruebas

Cuenta de GitHub: juniorbarba234
Fecha de verificación: 13 de octubre de 2026

Repositorio de esta tarea

https://github.com/juniorbarba234/API-Apartamentos

Este enlace corresponde a API Apartamentos. Cada una de las otras actividades se entrega con su propio repositorio.

Descripción

La API administra apartamentos y sus unidades de alojamiento. Relaciona al propietario, la propiedad, la unidad y el cliente para registrar reservas. Calcula el total por noche y permite llevar el registro de los pagos y de su estado.

Ruta	Qué administra
/api/clientes	Clientes y propietarios
/api/propiedades	Apartamentos y propiedades
/api/unidades	Unidades disponibles para reservar
/api/reservas	Fechas, cliente e importe total
/api/pagos	Importe, confirmación y reembolso
/api/disponibilidades	Periodos registrados
/api/resenas	Calificaciones y comentarios
/api/notificaciones	Mensajes guardados
/api/propiedad-imagenes	Enlaces de imágenes

Cómo se organiza

El controlador recibe la solicitud de Postman. El servicio revisa las reglas. El repositorio consulta o guarda los datos. El modelo representa datos como un cliente, una unidad o una reserva.

Pruebas realizadas con Postman y Newman

Se ejecutó la colección API Apartamentos con Newman, el ejecutor de colecciones de Postman. Las peticiones llegaron al servidor y sus respuestas se comprobaron automáticamente. Se utilizaron datos ficticios.

Entorno de esta ejecución: Java 26, Spring Boot 4.1.1 y H2 en memoria; dirección http://127.0.0.1:18080. La configuración habitual del proyecto utiliza MySQL. La conexión con el MySQL local queda por verificar después de configurar sus credenciales.

Verificación	Resultado
Solicitudes HTTP	40 de 40 aprobadas
Comprobaciones de respuesta	44 de 44 aprobadas
Pruebas de JUnit	8 de 8 aprobadas
Errores de la ejecución final	0

Una respuesta 400, 404 o 409 cuenta como prueba aprobada cuando se provocó ese error a propósito. Por ejemplo, el correo duplicado debe rechazarse con 409.

Archivos de evidencia

La colección importable está en la carpeta postman. La carpeta docs/pruebas contiene el informe de Newman, las respuestas recibidas en JSON y el resultado de Maven. Estos archivos permiten consultar los resultados y repetir las pruebas.

Abrir las evidencias del repositorio

Qué comprueban las pruebas

Se registran los nueve tipos de datos y se consultan sus respuestas. Se actualizan un cliente, una propiedad, una unidad y una reserva. También se elimina una imagen y se comprueba que ya no exista. Las validaciones rechazan correos duplicados, fechas incorrectas, reservas que se cruzan, clientes inexistentes y calificaciones fuera de rango.

En pagos se verifica el límite del importe, la confirmación, el reembolso y las operaciones que deben rechazarse después de confirmar un pago. En reservas se comprueba que el servidor calcule tres noches a 500 como un importe de 1500.

Resultados de las solicitudes 1 a 20

N	Prueba	Método	HTTP	Resultado
1	Crear propietario	POST	200	Aprobada
2	Crear cliente	POST	200	Aprobada
3	Rechazar correo duplicado	POST	409	Aprobada
4	Rechazar cliente sin nombre	POST	400	Aprobada
5	Crear propiedad	POST	200	Aprobada
6	Consultar propiedades por tipo	GET	200	Aprobada
7	Consultar propiedades por propietario	GET	200	Aprobada
8	Crear unidad rentable	POST	201	Aprobada
9	Reservar tres noches y calcular total	POST	201	Aprobada
10	Rechazar reserva duplicada	POST	400	Aprobada
11	Rechazar fechas invertidas	POST	400	Aprobada
12	Crear pago	POST	201	Aprobada
13	Rechazar pago que excede el total	POST	400	Aprobada
14	Confirmar pago	POST	200	Aprobada
15	Rechazar segunda confirmación	POST	400	Aprobada
16	Rechazar eliminación de pago confirmado	DELETE	400	Aprobada
17	Crear disponibilidad	POST	201	Aprobada
18	Registrar reseña	POST	201	Aprobada
19	Registrar notificación	POST	201	Aprobada
20	Registrar imagen	POST	201	Aprobada

Resultados de las solicitudes 21 a 40

N	Prueba	Método	HTTP	Resultado
21	Consultar clientes	GET	200	Aprobada
22	Consultar propiedades	GET	200	Aprobada
23	Consultar unidades	GET	200	Aprobada
24	Consultar reservas	GET	200	Aprobada
25	Consultar pagos	GET	200	Aprobada
26	Consultar disponibilidades	GET	200	Aprobada
27	Consultar resenas	GET	200	Aprobada
28	Consultar notificaciones	GET	200	Aprobada
29	Consultar propiedad-imagenes	GET	200	Aprobada
30	Actualizar unidad	PUT	200	Aprobada
31	Actualizar cliente	PUT	200	Aprobada
32	Actualizar propiedad	PUT	200	Aprobada
33	Actualizar reserva	PUT	200	Aprobada
34	Rechazar edición de pago confirmado	PUT	400	Aprobada
35	Reembolsar pago	POST	200	Aprobada
36	Rechazar reserva con cliente inexistente	POST	400	Aprobada
37	Rechazar reseña fuera de rango	POST	400	Aprobada
38	Consultar identificador inexistente	GET	404	Aprobada
39	Eliminar imagen de prueba	DELETE	204	Aprobada
40	Confirmar eliminación de imagen	GET	404	Aprobada

Ejemplo de una reserva

Primero se registra al propietario, después la propiedad, la unidad y el cliente. La colección guarda sus identificadores para utilizarlos en las solicitudes siguientes.

Solicitud: POST /api/reservas
Este ejemplo utiliza la unidad 1 y el cliente 2 generados durante la prueba.

```
{
  "unidad": {"idUnidad": 1},
  "cliente": {"id": 2},
  "fechaEntrada": "2026-11-01T15:00:00",
  "fechaSalida": "2026-11-04T11:00:00",
  "estado": "PENDIENTE",
  "importeTotal": 1
}
```

Respuesta recibida: HTTP 201. Extracto de los campos de la reserva:

```
{
  "idReservacion": 1,
  "fechaEntrada": "2026-11-01T15:00:00",
  "fechaSalida": "2026-11-04T11:00:00",
  "importeTotal": 1500.0,
  "estado": "PENDIENTE"
}
```

La unidad cuesta 500 por noche. Del 1 al 4 de noviembre son tres noches: 3 × 500 = 1500. Aunque se envió un importe de 1, el servidor calculó el precio con los datos guardados de la unidad.

Ejemplo de una validación

Al enviar otra reserva para la misma unidad y las mismas fechas, el servidor devuelve HTTP 400. Extracto de la respuesta real:

```
{
  "status": 400,
  "error": "La unidad ya está reservada en ese rango"
}
```

Pruebas de pagos

Se registró un pago de 1500 y se confirmó. Se rechazaron un pago adicional que excedía el importe, la segunda confirmación y la eliminación del pago confirmado. Finalmente se reembolsó el pago mediante su operación específica.

Cómo iniciar el proyecto

1. Iniciar MySQL.
2. Abrir CONFIGURAR.cmd con doble clic. Escribir el usuario y la contraseña de MySQL y pulsar Guardar configuración.
3. Abrir EJECUTAR.cmd con doble clic. Esperar el mensaje Started ApartamentosApplication y dejar la ventana abierta.

El proyecto necesita Java 21 o superior. La contraseña de MySQL se guarda en un archivo local excluido de GitHub. Las pantallas incluidas en la base se abren en http://localhost:8000.

Cómo repetir las pruebas en Postman

1. Abrir Postman y pulsar Import.
2. Seleccionar API Apartamentos.postman_collection.json dentro de la carpeta postman.
3. Verificar que baseUrl sea http://localhost:8000.
4. Ejecutar la colección en orden con Run collection, o enviar las solicitudes una por una empezando por Crear propietario.
5. Revisar los resultados. La colección guarda los identificadores automáticamente.

Cómo explicar el proyecto

“Mi API registra propietarios, apartamentos y clientes. Cada apartamento tiene unidades que se pueden reservar. Una reserva une al cliente con la unidad y sus fechas. El sistema calcula el total por noche y evita reservas con fechas que se cruzan. También registra pagos y permite confirmarlos o reembolsarlos.”

“Postman envía una solicitud y muestra la respuesta. GET consulta, POST crea, PUT actualiza y DELETE elimina.”

Alcance del proyecto

Los pagos son registros de la aplicación y no realizan cobros bancarios. Las notificaciones son mensajes guardados y no envían correos. La disponibilidad se administra por separado y las reservas utilizan el precio habitual de la unidad. El proyecto académico no incluye inicio de sesión ni controles de acceso. La ejecución documentada verifica solicitudes secuenciales.

Entrega de API Apartamentos:
https://github.com/juniorbarba234/API-Apartamentos